

Sistema RAG sobre Google Drive

Grupo Contábil Bastos & Luz — Especificação Técnica e Código de Referência

Maio de 2026

Sumário Executivo

Este documento especifica e implementa um sistema de **busca inteligente** sobre o acervo documental de um escritório de contabilidade hospedado no Google Drive (~50 GB, 332 pastas-cliente, formatos PDF/DOCX/XLSX).

A interação acontece por dois canais:

- **WhatsApp** — funcionário envia perguntas em linguagem natural (“qual o CNPJ da Padaria do João?”) e o bot responde com base nos documentos do cliente.
- **Dashboard web** — interface para navegar, visualizar e editar arquivos do Drive in-place, além de consultar a IA.

A solução usa arquitetura **RAG (Retrieval-Augmented Generation)**: documentos são indexados em embeddings vetoriais (Google `text-embedding-004`), armazenados em PostgreSQL com extensão `pgvector`, e consultas combinam busca semântica com o modelo **Gemini 2.5 Flash** para resposta natural com citação de fonte.

Escopo confirmado:

- 2 usuários internos (acesso total)
- 332 clientes (1 pasta = 1 cliente, nome da pasta = razão social)
- Documentos majoritariamente imutáveis, com adições diárias
- VPS única (Hostinger, Ubuntu 24.04, IP `168.231.68.93`)

1. Visão Geral

1.1. Fluxo de uso

Pelo WhatsApp:

```
Funcionário: "qual o cnpj da padaria do joão?"

Bot:  Encontrei: *Padaria do João Ltda - ME*
      Buscando nos documentos...

Bot:  CNPJ: 12.345.678/0001-90
      Fonte: Contrato Social 2024.pdf (página 1)
      [Abrir no Drive ➤]
```

Pelo Dashboard:

- Busca de pastas com fuzzy match
- Lista de arquivos por pasta com filtros (PDF/DOC/XLSX)
- Preview inline via iframe nativo do Drive
- Botão “Abrir no Google” para edição nativa
- Upload com indexação automática
- Painel de perguntas com histórico

1.2. Arquitetura em alto nível



1.3. Componentes lógicos

Componente	Responsabilidade
Sync Worker	Lê o Drive periodicamente, popula tabela <code>Cliente / DriveFile</code>
Indexer Worker	Processa arquivos pendentes: extração → chunk → embedding → pgvector
OCR Worker	Pipeline separado para PDFs escaneados (Google Cloud Vision)
API REST	Endpoints de consulta, listagem, upload, auth
WhatsApp Service	Baileys + roteamento de mensagens pro endpoint de consulta
Frontend React	Dashboard de pastas, arquivos, consulta e fila
Nginx	TLS, reverse proxy, rate limiting, static

2. Stack Tecnológico

Camada	Tecnologia	Justificativa
Frontend	React 18 + Vite + TypeScript	Tipagem para reduzir bugs em produção
Backend	Node.js 20 + Express + TypeScript	Stack único, time pequeno
ORM	Prisma 5	Migrations versionadas, type-safe
Banco	PostgreSQL 16	Já em uso, suporta pgvector + pg_trgm
Busca vetorial	pgvector 0.7+	Evita serviço externo (Pinecone etc.)
Fuzzy match	pg_trgm	Identificação de cliente por nome aproximado
IA generativa	Gemini 2.5 Flash	Custo-benefício para RAG simples
Embeddings	text-embedding-004	Português nativo, 768 dims, barato
OCR	Google Cloud Vision	Qualidade alta em PT-BR
WhatsApp	Baileys (provisório)	Migrar para Cloud API oficial no futuro
Process Manager	PM2	Já em uso, cluster mode
Proxy	Nginx	Já em uso
TLS	Let's Encrypt + Certbot	Gratuito, renovação automática

3. Estrutura de Diretórios

```
saas_financeiro/
├── backend/
│   ├── src/
│   │   ├── index.ts                # Entry point Express
│   │   ├── config/
│   │   │   ├── env.ts              # Validação de variáveis
│   │   │   └── google.ts           # Auth Drive + Vision + Gemini
│   │   ├── db/
│   │   │   └── prisma.ts            # Singleton Prisma client
│   │   ├── middleware/
│   │   │   ├── auth.ts              # JWT verify
│   │   │   ├── mfa.ts               # TOTP verify
│   │   │   └── ratelimit.ts          # Rate limiting Express
│   │   ├── services/
│   │   │   ├── drive.ts              # Wrapper Drive API
│   │   │   ├── gemini.ts             # Wrapper Gemini + embeddings
│   │   │   ├── ocr.ts                # Wrapper Cloud Vision
│   │   │   ├── extractor.ts           # PDF/DOCX/XLSX → texto
│   │   │   ├── chunker.ts            # Quebra de texto
│   │   │   ├── rag.ts                # Pipeline de consulta
│   │   │   ├── clienteResolver.ts     # Fuzzy match de nome
│   │   │   └── whatsapp.ts           # Baileys
│   │   ├── workers/
│   │   │   ├── syncDrive.ts          # Cron de sync
│   │   │   ├── indexFiles.ts         # Worker de indexação
│   │   │   └── ocrPending.ts          # Worker de OCR
│   │   └── routes/
│   │       ├── auth.ts
│   │       ├── clientes.ts
│   │       ├── files.ts
│   │       ├── ask.ts
│   │       └── queries.ts
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── migrations/
│   ├── scripts/
│   │   ├── sample-ocr.ts             # Amostragem de PDFs escaneados
│   │   ├── seed-clientes.ts          # Popula 332 clientes do Drive
│   │   └── backup.sh                  # Backup do Postgres
│   ├── .env.example
│   ├── package.json
│   └── tsconfig.json
├── frontend/
│   ├── src/
│   │   ├── main.tsx
│   │   ├── App.tsx
│   │   ├── api/
│   │   │   └── client.ts              # Axios + auth
│   │   ├── components/
│   │   │   ├── ClienteList.tsx
│   │   │   ├── FileExplorer.tsx
│   │   │   ├── FilePreview.tsx
│   │   │   ├── AskPanel.tsx
│   │   │   └── LoginForm.tsx
│   │   ├── pages/
│   │   │   ├── Dashboard.tsx
│   │   │   ├── Login.tsx
│   │   │   └── Cliente.tsx
│   │   └── styles/
│   └── package.json
├── nginx/
│   └── saas_financeiro.conf
├── deploy/
│   ├── deploy.sh
│   └── ecosystem.config.js            # PM2
```

4. Setup do Ambiente

4.1. Variáveis de ambiente

Arquivo `backend/.env.example` (publicar versionado; o `.env` real fica fora do git, com `chmod 600`):

```
# Banco
DATABASE_URL="postgresql://saas_admin:SENHA@localhost:5432/saas_financeiro"

# Aplicação
NODE_ENV=production
PORT=5000
PUBLIC_URL=https://portal.bastoseluz.com.br

# Auth
JWT_SECRET=                # openssl rand -hex 64
JWT_EXPIRES_IN=2h
REFRESH_TOKEN_EXPIRES_IN=14d
COOKIE_SECURE=true
COOKIE_SAMESITE=strict

# Google (mesma service account para Drive, Vision e Gemini)
GOOGLE_APPLICATION_CREDENTIALS=/root/saas_financeiro/secrets/gcp-sa.json
GOOGLE_DRIVE_ROOT_FOLDER_ID=    # ID da pasta raiz "Clientes"
GEMINI_API_KEY=
GEMINI_MODEL=gemini-2.5-flash
EMBEDDING_MODEL=text-embedding-004

# WhatsApp
WHATSAPP_WHITELIST=5592999990001,5592999990002

# Backup
BACKUP_GPG_PASSPHRASE_FILE=/root/.backup_key
BACKUP_REMOTE=b2:bastoseluz-backups
```

4.2. Habilitando extensões do Postgres

Conecte como superuser e execute:

```
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS unaccent; -- busca insensível a acentos
```

4.3. Dependências do backend

`backend/package.json` :

```

{
  "name": "saas-financeiro-backend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js",
    "worker:sync": "tsx src/workers/syncDrive.ts",
    "worker:index": "tsx src/workers/indexFiles.ts",
    "worker:ocr": "tsx src/workers/ocrPending.ts",
    "sample:ocr": "tsx scripts/sample-ocr.ts",
    "seed:clientes": "tsx scripts/seed-clientes.ts",
    "prisma:migrate": "prisma migrate deploy",
    "prisma:generate": "prisma generate"
  },
  "dependencies": {
    "@baileys/baileys": "^6.7.0",
    "@google-cloud/vision": "^4.3.2",
    "@google/generative-ai": "^0.21.0",
    "@prisma/client": "^5.20.0",
    "bcrypt": "^5.1.1",
    "cookie-parser": "^1.4.7",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.21.0",
    "express-rate-limit": "^7.4.0",
    "googleapis": "^144.0.0",
    "helmet": "^8.0.0",
    "jsonwebtoken": "^9.0.2",
    "mammoth": "^1.8.0",
    "otplib": "^12.0.1",
    "pdf-parse": "^1.1.1",
    "pino": "^9.4.0",
    "qrcode": "^1.5.4",
    "xlsx": "^0.18.5",
    "zod": "^3.23.8"
  },
  "devDependencies": {
    "@types/bcrypt": "^5.0.2",
    "@types/cookie-parser": "^1.4.7",
    "@types/cors": "^2.8.17",
    "@types/express": "^4.17.21",
    "@types/jsonwebtoken": "^9.0.7",
    "@types/node": "^22.7.4",
    "@types/pdf-parse": "^1.1.4",
    "prisma": "^5.20.0",
    "tsx": "^4.19.1",
    "typescript": "^5.6.2"
  }
}

```

5. Schema do Banco de Dados

Arquivo `backend/prisma/schema.prisma`:

```
generator client {
  provider      = "prisma-client-js"
  previewFeatures = ["postgresqlExtensions"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  extensions = [pgvector(map: "vector"), pg_trgm, unaccent]
}

// =====
// USUÁRIOS INTERNOS (apenas 2 funcionários)
// =====

model AppUser {
  id          String      @id @default(uuid())
  email       String      @unique
  phoneE164   String      @unique // whitelist WhatsApp
  name        String
  passwordHash String
  totpSecret  String?
  isActive    Boolean      @default(true)
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt
  queries     QueryLog[]
  sessions    UserSession[]
}

model UserSession {
  id          String      @id @default(uuid())
  userId      String
  user        AppUser     @relation(fields: [userId], references: [id], onDelete: Cascade)
  refreshToken String      @unique
  expiresAt   DateTime
  revokedAt   DateTime?
  userAgent   String?
  ipAddress   String?
  createdAt   DateTime @default(now())
  @@index([userId])
}

// =====
// ESTRUTURA DO DRIVE (332 clientes)
// =====

model Cliente {
  id          String      @id @default(uuid())
  driveFolderId String      @unique
  nomeEmpresa String
  cnpj        String?      // preenchido via IA quando descoberto
  isActive    Boolean      @default(true)
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt
  files       DriveFile[]
  queries     QueryLog[]

  @@index([nomeEmpresa])
}

model DriveFile {
  id          String      @id @default(uuid())
  driveFileId String      @unique
  fileName    String
  mimeType    String
  sizeBytes   BigInt
  modifiedTime DateTime
  webViewLink String?      // link público pro Drive
  clienteId   String
  cliente     Cliente      @relation(fields: [clienteId], references: [id], onDelete: Cascade)
  indexStatus IndexStatus  @default(PENDING)
  isScanned   Boolean?
  errorMessage String?
  contentHash String?      // sha256 para detectar mudança real
  pageCount   Int?
  indexedAt   DateTime?
}
```

```

    createdAt    DateTime    @default(now())
    updatedAt    DateTime    @updatedAt
    chunks       DocChunk[]

    @@index([clienteId, indexStatus])
    @@index([indexStatus, createdAt])
  }

  model DocChunk {
    id           String      @id @default(uuid())
    driveFileId  String
    driveFile    DriveFile   @relation(fields: [driveFileId], references: [id], onDelete: Cascade)
    clienteId    String      // denormalizado pra acelerar filtro
    content      String      @db.Text
    // embedding 768 dims (text-embedding-004)
    // Prisma não tem suporte nativo a vector ainda -> migration manual
    pageNumber  Int?
    chunkIndex   Int
    createdAt    DateTime    @default(now())

    @@index([clienteId])
    @@index([driveFileId])
  }

  // =====
  // AUDITORIA / LOGS
  // =====

  model QueryLog {
    id           String      @id @default(uuid())
    userId       String
    user         AppUser     @relation(fields: [userId], references: [id])
    clienteId    String?
    cliente      Cliente?    @relation(fields: [clienteId], references: [id])
    question     String      @db.Text
    answer       String      @db.Text
    sources      Json        // [{driveFileId, fileName, score, pageNumber}]
    channel      QueryChannel
    latencyMs    Int
    tokensIn     Int?
    tokensOut    Int?
    errorMsg     String?
    createdAt    DateTime    @default(now())

    @@index([userId, createdAt])
    @@index([clienteId, createdAt])
  }

  model DriveSyncState {
    id           Int         @id @default(1) // singleton
    pageToken    String?    // changes.list cursor
    lastSyncAt   DateTime    @default(now())
  }

  // =====
  // ENUMS
  // =====

  enum IndexStatus {
    PENDING      // recém descoberto
    EXTRACTING   // baixando + extraindo texto
    NEEDS_OCR    // PDF escaneado, vai pro pipeline OCR
    EMBEDDING    // criando vetores
    INDEXED      // pronto, busca habilitada
    FAILED       // erro -> ver errorMessage
    SKIPPED      // tipo não suportado
  }

  enum QueryChannel {
    WHATSAPP
    DASHBOARD
  }

```

5.1. Migration manual para o campo vetorial

O Prisma 5 ainda não tem suporte nativo a `vector(N)`. A migration deve ser editada à mão depois de `prisma migrate dev --create-only`:

Arquivo `backend/prisma/migrations/XXXX_add_vector_column/migration.sql`:


```

-- Migration gerada pelo Prisma + edição manual

-- Cria a tabela DocChunk normalmente (gerada pelo Prisma)
-- Depois adiciona a coluna vetorial:

ALTER TABLE "DocChunk"
  ADD COLUMN "embedding" vector(768);

-- Índice HNSW para busca por similaridade (cosine)
-- Aguenta centenas de milhares de chunks com latência ~ms.
CREATE INDEX docchunk_embedding_hnsw_idx
  ON "DocChunk"
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Índice GIN para fuzzy match em nomes de cliente
CREATE INDEX cliente_nome_trgm_idx
  ON "Cliente"
  USING gin (nomeEmpresa gin_trgm_ops);

-- Idem para fileName (busca de arquivo no dashboard)
CREATE INDEX drivefile_filename_trgm_idx
  ON "DriveFile"
  USING gin ("fileName" gin_trgm_ops);

```

Depois disso, rode no servidor:

```

npx prisma migrate deploy
npx prisma generate

```

6. Código Base do Backend

6.1. Validação de variáveis de ambiente

backend/src/config/env.ts :

```
import { z } from 'zod';
import 'dotenv/config';

const envSchema = z.object({
  NODE_ENV: z.enum(['development', 'production']).default('production'),
  PORT: z.coerce.number().default(5000),
  PUBLIC_URL: z.string().url(),
  DATABASE_URL: z.string().url(),

  JWT_SECRET: z.string().min(32),
  JWT_EXPIRES_IN: z.string().default('2h'),
  REFRESH_TOKEN_EXPIRES_IN: z.string().default('14d'),
  COOKIE_SECURE: z.coerce.boolean().default(true),
  COOKIE_SAMESITE: z.enum(['strict', 'lax', 'none']).default('strict'),

  GOOGLE_APPLICATION_CREDENTIALS: z.string(),
  GOOGLE_DRIVE_ROOT_FOLDER_ID: z.string(),
  GEMINI_API_KEY: z.string(),
  GEMINI_MODEL: z.string().default('gemini-2.5-flash'),
  EMBEDDING_MODEL: z.string().default('text-embedding-004'),

  WHATSAPP_WHITELIST: z.string().transform((s) =>
    s.split(',').map((x) => x.trim()).filter(Boolean),
  ),
});

const parsed = envSchema.safeParse(process.env);
if (!parsed.success) {
  console.error(' Variáveis de ambiente inválidas:', parsed.error.format());
  process.exit(1);
}

export const env = parsed.data;
```

6.2. Cliente Prisma singleton

backend/src/db/prisma.ts :

```
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    log: process.env.NODE_ENV === 'development' ? ['query', 'error'] : ['error'],
  });

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

6.3. Auth do Google (Drive + Vision)

backend/src/config/google.ts :

```
import { google } from 'googleapis';
import { ImageAnnotatorClient } from '@google-cloud/vision';
import { GoogleGenerativeAI } from '@google/generative-ai';
import { env } from './env.js';

// Drive API - usa Application Default Credentials (arquivo da service account)
const auth = new google.auth.GoogleAuth({
  scopes: ['https://www.googleapis.com/auth/drive'],
});

export const driveClient = google.drive({ version: 'v3', auth });
export const visionClient = new ImageAnnotatorClient();
export const geminiClient = new GoogleGenerativeAI(env.GEMINI_API_KEY);

export const geminiModel = geminiClient.getGenerativeModel({
  model: env.GEMINI_MODEL,
  generationConfig: { temperature: 0.1 }, // determinístico para RAG
});

export const embeddingModel = geminiClient.getGenerativeModel({
  model: env.EMBEDDING_MODEL,
});
```

7. Serviços de Negócio

7.1. Serviço de Drive

backend/src/services/drive.ts:

```
import { driveClient } from '../config/google.js';
import { Readable } from 'stream';

export interface DriveFileInfo {
  id: string;
  name: string;
  mimeType: string;
  size: string;
  modifiedTime: string;
  parents?: string[];
  webViewLink?: string;
}

/**
 * Lista as pastas-cliente filhas da raiz.
 */
export async function listClientFolders(rootFolderId: string) {
  const folders: DriveFileInfo[] = [];
  let pageToken: string | undefined;

  do {
    const res = await driveClient.files.list({
      q: ` '${rootFolderId}' in parents and mimeType='application/vnd.google-apps.folder' and trashed=false`,
      fields: 'nextPageToken, files(id, name, modifiedTime)',
      pageSize: 1000,
      pageToken,
    });
    folders.push(...((res.data.files ?? []) as DriveFileInfo[]));
    pageToken = res.data.nextPageToken ?? undefined;
  } while (pageToken);

  return folders;
}

/**
 * Lista todos os arquivos dentro de uma pasta (recursivo opcional).
 */
export async function listFilesInFolder(folderId: string, recursive = true) {
  const files: DriveFileInfo[] = [];
  const stack = [folderId];

  while (stack.length) {
    const current = stack.pop()!;
    let pageToken: string | undefined;

    do {
      const res = await driveClient.files.list({
        q: ` '${current}' in parents and trashed=false`,
        fields: 'nextPageToken, files(id, name, mimeType, size, modifiedTime, parents, webViewLink)',
        pageSize: 1000,
        pageToken,
      });

      for (const f of res.data.files ?? []) {
        if (f.mimeType === 'application/vnd.google-apps.folder') {
          if (recursive) stack.push(f.id!);
        } else {
          files.push(f as DriveFileInfo);
        }
      }
      pageToken = res.data.nextPageToken ?? undefined;
    } while (pageToken);
  }

  return files;
}

/**
 * Baixa o conteúdo de um arquivo como Buffer.
 * Para Google Docs/Sheets nativos, exporta como DOCX/XLSX.
 */
```

```

export async function downloadFile(fileId: string, mimeType: string): Promise<Buffer> {
  const GOOGLE_DOC = 'application/vnd.google-apps.document';
  const GOOGLE_SHEET = 'application/vnd.google-apps.spreadsheet';

  if (mimeType === GOOGLE_DOC) {
    const res = await driveClient.files.export(
      {
        fileId,
        mimeType:
          'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
      },
      { responseType: 'arraybuffer' },
    );
    return Buffer.from(res.data as ArrayBuffer);
  }

  if (mimeType === GOOGLE_SHEET) {
    const res = await driveClient.files.export(
      {
        fileId,
        mimeType:
          'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
      },
      { responseType: 'arraybuffer' },
    );
    return Buffer.from(res.data as ArrayBuffer);
  }

  const res = await driveClient.files.get(
    { fileId, alt: 'media' },
    { responseType: 'arraybuffer' },
  );
  return Buffer.from(res.data as ArrayBuffer);
}

/**
 * Detecta mudanças desde o último token salvo.
 * Documentação: https://developers.google.com/drive/api/guides/manage-changes
 */
export async function getChanges(pageToken: string) {
  const changes: any[] = [];
  let currentToken: string | undefined = pageToken;
  let newStartPageToken: string | undefined;

  do {
    const res = await driveClient.changes.list({
      pageToken: currentToken,
      pageSize: 1000,
      fields:
        'nextPageToken, newStartPageToken, changes(fileId, removed, file(id, name, mimeType, size, modifiedTime, parents, webViewLink))',
    });
    changes.push(...(res.data.changes ?? []));
    currentToken = res.data.nextPageToken ?? undefined;
    if (res.data.newStartPageToken) newStartPageToken = res.data.newStartPageToken;
  } while (currentToken);

  return { changes, newStartPageToken };
}

export async function getStartPageToken() {
  const res = await driveClient.changes.getStartPageToken();
  return res.data.startPageToken!;
}

```

8. Pipeline de Extração e Indexação

8.1. Extrator multi-formato

backend/src/services/extractor.ts:

```
import pdfParse from 'pdf-parse';
import mammoth from 'mammoth';
import * as XLSX from 'xlsx';

export interface ExtractedPage {
  pageNumber?: number;
  text: string;
}

export interface ExtractionResult {
  pages: ExtractedPage[];
  totalText: string;
  needsOcr: boolean;
  pageCount: number;
}

const MIN_TEXT_PER_PAGE = 50; // < 50 chars/página = provavelmente escaneado

/**
 * Extrai texto preservando estrutura por página (quando possível).
 */
export async function extract(
  buffer: Buffer,
  mimeType: string,
  fileName: string,
): Promise<ExtractionResult> {
  if (mimeType === 'application/pdf' || fileName.toLowerCase().endsWith('.pdf')) {
    return extractPdf(buffer);
  }
  if (
    mimeType.includes('wordprocessingml') ||
    fileName.toLowerCase().endsWith('.docx')
  ) {
    return extractDocx(buffer);
  }
  if (
    mimeType.includes('spreadsheetml') ||
    fileName.toLowerCase().endsWith('.xlsx') ||
    fileName.toLowerCase().endsWith('.xls')
  ) {
    return extractXlsx(buffer);
  }
  throw new Error(`Formato não suportado: ${mimeType} (${fileName})`);
}

async function extractPdf(buffer: Buffer): Promise<ExtractionResult> {
  const data = await pdfParse(buffer);
  const pageCount = data.numpages;
  // pdf-parse retorna todo o texto junto, sem separação clara por página
  // Para separação real por página, usar pdfjs-dist (mais complexo)
  const text = data.text.trim();
  const avgPerPage = text.length / Math.max(pageCount, 1);
  const needsOcr = avgPerPage < MIN_TEXT_PER_PAGE;

  return {
    pages: needsOcr ? [] : [{ text }],
    totalText: text,
    needsOcr,
    pageCount,
  };
}

async function extractDocx(buffer: Buffer): Promise<ExtractionResult> {
  const result = await mammoth.extractRawText({ buffer });
  return {
    pages: [{ text: result.value }],
    totalText: result.value,
    needsOcr: false,
    pageCount: 1,
  };
}
```

```

async function extractXlsx(buffer: Buffer): Promise<ExtractionResult> {
  const workbook = XLSX.read(buffer, { type: 'buffer' });
  const pages: ExtractedPage[] = [];

  workbook.SheetNames.forEach((sheetName, idx) => {
    const sheet = workbook.Sheets[sheetName];
    // Converte para CSV preservando cabeçalho
    const csv = XLSX.utils.sheet_to_csv(sheet, { blankrows: false });
    // Anota a aba no texto para o RAG ter contexto
    const annotated = `[Planilha: ${sheetName}]\n${csv}`;
    pages.push({ pageNumber: idx + 1, text: annotated });
  });

  const totalText = pages.map((p) => p.text).join('\n\n');
  return {
    pages,
    totalText,
    needsOcr: false,
    pageCount: pages.length,
  };
}

```

8.2. Chunker

backend/src/services/chunker.ts :

```

const CHUNK_SIZE = 800;           // tokens aproximados (~3200 caracteres em PT)
const OVERLAP = 100;             // tokens de sobreposição

// Estimativa simples: 1 token = 4 caracteres em português
const CHARS_PER_CHUNK = CHUNK_SIZE * 4;
const CHARS_OVERLAP = OVERLAP * 4;

export interface Chunk {
  content: string;
  pageNumber?: number;
  chunkIndex: number;
}

/**
 * Quebra texto em chunks com overlap, tentando respeitar fronteiras de
 * parágrafo (\n\n) e depois sentença (.) para preservar contexto.
 */
export function chunkText(text: string, pageNumber?: number, startIndex = 0): Chunk[] {
  const clean = text.replace(/s+\n/g, '\n').trim();
  if (clean.length === 0) return [];

  const chunks: Chunk[] = [];
  let cursor = 0;
  let idx = startIndex;

  while (cursor < clean.length) {
    let end = Math.min(cursor + CHARS_PER_CHUNK, clean.length);

    if (end < clean.length) {
      // tenta cortar em quebra de parágrafo
      const paragraphCut = clean.lastIndexOf('\n\n', end);
      if (paragraphCut > cursor + CHARS_PER_CHUNK / 2) {
        end = paragraphCut;
      } else {
        // ou em ponto final
        const sentenceCut = clean.lastIndexOf('.', end);
        if (sentenceCut > cursor + CHARS_PER_CHUNK / 2) {
          end = sentenceCut + 1;
        }
      }
    }

    const slice = clean.slice(cursor, end).trim();
    if (slice.length > 0) {
      chunks.push({ content: slice, pageNumber, chunkIndex: idx++ });
    }

    cursor = end - CHARS_OVERLAP;
    if (cursor <= 0 || cursor >= clean.length) break;
  }

  return chunks;
}

```

8.3. Wrapper de embeddings

backend/src/services/gemini.ts :

```
import { embeddingModel, geminiModel } from '../config/google.js';

/**
 * Gera embedding para um único texto.
 */
export async function embed(text: string): Promise<number[]> {
  const result = await embeddingModel.embedContent(text);
  return result.embedding.values;
}

/**
 * Gera embeddings em batch (mais eficiente).
 * text-embedding-004 aceita até 100 documentos por chamada.
 */
export async function embedBatch(texts: string[]): Promise<number[][]> {
  const batches: string[][] = [];
  for (let i = 0; i < texts.length; i += 100) {
    batches.push(texts.slice(i, i + 100));
  }

  const all: number[][] = [];
  for (const batch of batches) {
    const result = await embeddingModel.batchEmbedContents({
      requests: batch.map((text) => ({
        content: { role: 'user', parts: [{ text }] },
      })),
    });
    all.push(...result.embeddings.map((e) => e.values));
  }
  return all;
}

/**
 * Pergunta + contexto recuperado -> resposta natural.
 */
export async function ask(question: string, context: string): Promise<string> {
  const prompt = `Você é um assistente do escritório contábil Bastos & Luz.
  Responda à pergunta usando APENAS as informações do contexto abaixo.
  Se a resposta não estiver no contexto, diga "Não encontrei essa informação nos documentos do cliente."
  Seja conciso e direto. Sempre cite o nome do arquivo de origem.

  CONTEXT0:
  ${context}

  PERGUNTA: ${question}

  RESPOSTA:`;

  const result = await geminiModel.generateContent(prompt);
  return result.response.text();
}

/**
 * Extraí o nome de empresa que o usuário citou na pergunta.
 * Retorna null se não conseguir identificar.
 */
export async function extractCompanyName(question: string): Promise<string | null> {
  const prompt = `Extraia o nome da empresa/cliente citado na pergunta abaixo.
  Responda APENAS com o nome, sem aspas, sem comentários.
  Se não houver empresa citada, responda exatamente: NONE

  Pergunta: ${question}

  Empresa:`;

  const result = await geminiModel.generateContent(prompt);
  const text = result.response.text().trim();
  if (text === 'NONE' || text.length === 0) return null;
  return text;
}
```

8.4. OCR com Cloud Vision

backend/src/services/ocr.ts :


```
import { visionClient } from '../config/google.js';
import { ExtractedPage } from './extractor.js';

/**
 * Executa OCR em um PDF escaneado via Cloud Vision.
 * Cloud Vision aceita PDF diretamente até 2000 páginas / 20MB por chamada.
 */
export async function ocrPdf(buffer: Buffer): Promise<ExtractedPage[]> {
  const request = {
    requests: [
      {
        inputConfig: {
          content: buffer.toString('base64'),
          mimeType: 'application/pdf',
        },
        features: [{ type: 'DOCUMENT_TEXT_DETECTION' as const }],
        imageContext: { languageHints: ['pt-BR'] },
      },
    ],
  };

  const [result] = await visionClient.batchAnnotateFiles(request);
  const responses = result.responses?.[0]?.responses ?? [];

  return responses.map((r, idx) => ({
    pageNumber: idx + 1,
    text: r.fullTextAnnotation?.text ?? '',
  }));
}
```

9. Workers de Background

9.1. Sync do Drive

Roda a cada 15 minutos via cron. Detecta arquivos novos e modificados.

`backend/src/workers/syncDrive.ts`:

```
import { prisma } from '../db/prisma.js';
import {
  listClientFolders,
  listFilesInFolder,
  getChanges,
  getStartPageToken,
} from '../services/drive.js';
import { env } from '../config/env.js';

const SUPPORTED_MIMES = [
  'application/pdf',
  'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
  'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
  'application/vnd.google-apps.document',
  'application/vnd.google-apps.spreadsheet',
  'application/msword',
  'application/vnd.ms-excel',
];

async function fullSync() {
  console.log(' Sync completo iniciado');

  const folders = await listClientFolders(env.GOOGLE_DRIVE_ROOT_FOLDER_ID);
  console.log(` ${folders.length} pastas-cliente encontradas`);

  for (const folder of folders) {
    const cliente = await prisma.cliente.upsert({
      where: { driveFolderId: folder.id },
      update: { nomeEmpresa: folder.name },
      create: { driveFolderId: folder.id, nomeEmpresa: folder.name },
    });

    const files = await listFilesInFolder(folder.id);
    let novos = 0;

    for (const file of files) {
      if (!SUPPORTED_MIMES.includes(file.mimeType)) continue;

      const existing = await prisma.driveFile.findUnique({
        where: { driveFileId: file.id },
      });

      if (!existing) {
        await prisma.driveFile.create({
          data: {
            driveFileId: file.id,
            fileName: file.name,
            mimeType: file.mimeType,
            sizeBytes: BigInt(file.size ?? 0),
            modifiedTime: new Date(file.modifiedTime),
            webViewLink: file.webViewLink ?? null,
            clienteId: cliente.id,
            indexStatus: 'PENDING',
          },
        });
        novos++;
      } else if (
        existing.modifiedTime.getTime() !== new Date(file.modifiedTime).getTime()
      ) {
        // Arquivo foi alterado: marca para reindexação
        await prisma.driveFile.update({
          where: { id: existing.id },
          data: {
            modifiedTime: new Date(file.modifiedTime),
            indexStatus: 'PENDING',
            indexedAt: null,
          },
        });
      }
      await prisma.docChunk.deleteMany({
        where: { driveFileId: existing.id },
      });
    }
  }
}
```

```

    });
    novos++;
  }
}
if (novos > 0) console.log(`  ↳ ${folder.name}: ${novos} novos/alterados`);
}

// Salva token para deltas futuros
const startToken = await getStartPageToken();
await prisma.driveSyncState.upsert({
  where: { id: 1 },
  update: { pageToken: startToken, lastSyncAt: new Date() },
  create: { id: 1, pageToken: startToken },
});

console.log('  Sync completo finalizado');
}

async function incrementalSync() {
  const state = await prisma.driveSyncState.findUnique({ where: { id: 1 } });
  if (!state?.pageToken) {
    console.log('⚠ Sem pageToken, rodando sync completo');
    return fullSync();
  }

  console.log('  Sync incremental iniciado');
  const { changes, newStartPageToken } = await getChanges(state.pageToken);
  console.log(`  ${changes.length} mudanças detectadas`);

  for (const change of changes) {
    if (change.removed) {
      await prisma.driveFile.deleteMany({
        where: { driveFileId: change.fileId },
      });
      continue;
    }
    const file = change.file;
    if (!file || !SUPPORTED_MIMES.includes(file.mimeType)) continue;

    // Resolve cliente pela primeira pasta-pai conhecida
    const cliente = await prisma.cliente.findFirst({
      where: { driveFolderId: { in: file.parents ?? [] } },
    });
    if (!cliente) continue; // arquivo fora das pastas monitoradas

    await prisma.driveFile.upsert({
      where: { driveFileId: file.id },
      update: {
        fileName: file.name,
        modifiedTime: new Date(file.modifiedTime),
        sizeBytes: BigInt(file.size ?? 0),
        indexStatus: 'PENDING',
        indexedAt: null,
      },
      create: {
        driveFileId: file.id,
        fileName: file.name,
        mimeType: file.mimeType,
        sizeBytes: BigInt(file.size ?? 0),
        modifiedTime: new Date(file.modifiedTime),
        webViewLink: file.webViewLink ?? null,
        clienteId: cliente.id,
        indexStatus: 'PENDING',
      },
    });
  }

  if (newStartPageToken) {
    await prisma.driveSyncState.update({
      where: { id: 1 },
      data: { pageToken: newStartPageToken, lastSyncAt: new Date() },
    });
  }

  console.log('  Sync incremental finalizado');
}

const mode = process.argv[2] ?? 'incremental';
(mode === 'full' ? fullSync() : incrementalSync())
  .catch((err) => {
    console.error('  Falha no sync:', err);
    process.exit(1);
  })
  .finally(() => prisma.$disconnect());

```

9.2. Worker de indexação

Pega arquivos **PENDING**, processa, popula chunks. Roda continuamente.

`backend/src/workers/indexFiles.ts`:

```
import { prisma } from '../db/prisma.js';
import { downloadFile } from '../services/drive.js';
import { extract } from '../services/extractor.js';
import { chunkText } from '../services/chunker.js';
import { embedBatch } from '../services/gemini.js';

const BATCH_SIZE = 5; // arquivos por ciclo
const POLL_INTERVAL_MS = 5000;

async function processOne(fileId: string) {
  const file = await prisma.driveFile.findUnique({ where: { id: fileId } });
  if (!file) return;

  try {
    await prisma.driveFile.update({
      where: { id: fileId },
      data: { indexStatus: 'EXTRACTING' },
    });

    console.log(` Baixando: ${file.fileName}`);
    const buffer = await downloadFile(file.driveFileId, file.mimeType);

    console.log(` Extraíndo texto: ${file.fileName}`);
    const result = await extract(buffer, file.mimeType, file.fileName);

    if (result.needsOcr) {
      console.log(` Marcando para OCR: ${file.fileName}`);
      await prisma.driveFile.update({
        where: { id: fileId },
        data: {
          indexStatus: 'NEEDS_OCR',
          isScanned: true,
          pageCount: result.pageCount,
        },
      });
      return;
    }

    // Quebra em chunks
    const chunks = result.pages.flatMap((page, pageIdx) =>
      chunkText(page.text, page.pageNumber ?? pageIdx + 1, 0),
    );

    if (chunks.length === 0) {
      await prisma.driveFile.update({
        where: { id: fileId },
        data: {
          indexStatus: 'SKIPPED',
          errorMessage: 'Texto vazio após extração',
        },
      });
      return;
    }

    await prisma.driveFile.update({
      where: { id: fileId },
      data: { indexStatus: 'EMBEDDING', pageCount: result.pageCount },
    });

    console.log(` Gerando ${chunks.length} embeddings: ${file.fileName}`);
    const vectors = await embedBatch(chunks.map((c) => c.content));

    // Insert em transação. Como o Prisma não suporta vector, usa SQL raw.
    await prisma.$transaction(
      chunks.map((chunk, i) =>
        prisma.$executeRaw`
          INSERT INTO "DocChunk"
            (id, "driveFileId", "clienteId", content, embedding, "pageNumber", "chunkIndex", "createdAt")
          VALUES (
            gen_random_uuid(),
            ${file.id},
            ${file.clienteId},
            ${chunk.content},
            ${['' + vectors[i].join(',') + '']::vector},
            ${chunk.pageNumber},
            ${i},
            now()
          )
        `
      )
    );
  } catch (error) {
    console.error('Erro ao processar arquivo:', error);
  }
}
```

```

        ${chunk.pageNumber ?? null},
        ${chunk.chunkIndex},
        NOW()
      ),
    ),
  );
};

await prisma.driveFile.update({
  where: { id: fileId },
  data: {
    indexStatus: 'INDEXED',
    indexedAt: new Date(),
    isScanned: false,
  },
});

console.log(`  Indexado: ${file.fileName} (${chunks.length} chunks)`);
} catch (err: any) {
  console.error(`  Falha em ${file.fileName}:`, err.message);
  await prisma.driveFile.update({
    where: { id: fileId },
    data: {
      indexStatus: 'FAILED',
      errorMessage: err.message?.slice(0, 500),
    },
  });
}
}
}

async function loop() {
  console.log('  Worker de indexação iniciado');

  while (true) {
    const pending = await prisma.driveFile.findMany({
      where: { indexStatus: 'PENDING' },
      take: BATCH_SIZE,
      orderBy: { createdAt: 'asc' },
    });

    if (pending.length === 0) {
      await new Promise((r) => setTimeout(r, POLL_INTERVAL_MS));
      continue;
    }

    await Promise.all(pending.map((f) => processOne(f.id)));
  }
}

loop().catch((err) => {
  console.error('Worker travou:', err);
  prisma.$disconnect();
  process.exit(1);
});

```

9.3. Worker de OCR

Pega arquivos marcados como `NEEDS_OCR`, processa via Cloud Vision.

`backend/src/workers/ocrPending.ts`:

```

import { prisma } from '../db/prisma.js';
import { downloadFile } from '../services/drive.js';
import { ocrPdf } from '../services/ocr.js';
import { chunkText } from '../services/chunker.js';
import { embedBatch } from '../services/gemini.js';

const POLL_INTERVAL_MS = 10000;

async function processOne(fileId: string) {
  const file = await prisma.driveFile.findUnique({ where: { id: fileId } });
  if (!file) return;

  try {
    console.log(`  OCR iniciado: ${file.fileName}`);
    const buffer = await downloadFile(file.driveFileId, file.mimeType);
    const pages = await ocrPdf(buffer);

    const chunks = pages.flatMap((page) =>
      chunkText(page.text, page.pageNumber, 0),
    );
  }
}

```

```

    });

    if (chunks.length === 0) {
      await prisma.driveFile.update({
        where: { id: fileId },
        data: {
          indexStatus: 'SKIPPED',
          errorMessage: 'OCR não encontrou texto',
        },
      });
      return;
    }

    const vectors = await embedBatch(chunks.map((c) => c.content));

    await prisma.$transaction(
      chunks.map((chunk, i) =>
        prisma.$executeRaw`
          INSERT INTO "DocChunk"
            (id, "driveFileId", "clienteId", content, embedding, "pageNumber", "chunkIndex", "createdAt")
          VALUES (
            gen_random_uuid(),
            ${file.id},
            ${file.clienteId},
            ${chunk.content},
            ${['' + vectors[i].join(',') + '']::vector},
            ${chunk.pageNumber ?? null},
            ${chunk.chunkIndex},
            NOW()
          )
        `
      ),
    );

    await prisma.driveFile.update({
      where: { id: fileId },
      data: { indexStatus: 'INDEXED', indexedAt: new Date() },
    });

    console.log(`  OCR concluído: ${file.fileName} (${chunks.length} chunks)`);
  } catch (err: any) {
    console.error(`  OCR falhou em ${file.fileName}:`, err.message);
    await prisma.driveFile.update({
      where: { id: fileId },
      data: { indexStatus: 'FAILED', errorMessage: err.message?.slice(0, 500) },
    });
  }
}

async function loop() {
  console.log('  Worker de OCR iniciado');
  while (true) {
    const pending = await prisma.driveFile.findFirst({
      where: { indexStatus: 'NEEDS_OCR' },
      orderBy: { createdAt: 'asc' },
    });
    if (!pending) {
      await new Promise((r) => setTimeout(r, POLL_INTERVAL_MS));
      continue;
    }
    await processOne(pending.id);
  }
}

loop().catch((err) => {
  console.error('Worker OCR travou:', err);
  prisma.$disconnect();
  process.exit(1);
});

```

10. Pipeline RAG

10.1. Resolução de cliente por nome

backend/src/services/clienteResolver.ts:

```
import { prisma } from '../db/prisma.js';
import { extractCompanyName } from './gemini.js';

export interface ClienteMatch {
  id: string;
  nomeEmpresa: string;
  driveFolderId: string;
  score: number;
}

export interface ResolveResult {
  status: 'resolved' | 'ambiguous' | 'not_found' | 'no_company_in_question';
  matches: ClienteMatch[];
}

const HIGH_CONFIDENCE = 0.7;
const MIN_GAP = 0.2; // diferença mínima entre top 1 e top 2

/**
 * Resolve o cliente que o usuário citou na pergunta.
 * Combina extração via Gemini + fuzzy match no Postgres via pg_trgm.
 */
export async function resolveCliente(question: string): Promise<ResolveResult> {
  const company = await extractCompanyName(question);
  if (!company) {
    return { status: 'no_company_in_question', matches: [] };
  }

  const rows = await prisma.$queryRaw<ClienteMatch[]>`
    SELECT
      id,
      "nomeEmpresa",
      "driveFolderId",
      similarity(unaccent(lower("nomeEmpresa")), unaccent(lower(${company}))) AS score
    FROM "Cliente"
    WHERE unaccent(lower("nomeEmpresa")) % unaccent(lower(${company}))
      AND "isActive" = true
    ORDER BY score DESC
    LIMIT 5
  `;

  if (rows.length === 0) {
    return { status: 'not_found', matches: [] };
  }

  const top = rows[0];
  const second = rows[1];

  if (
    top.score >= HIGH_CONFIDENCE &&
    (!second || top.score - second.score >= MIN_GAP)
  ) {
    return { status: 'resolved', matches: [top] };
  }

  return { status: 'ambiguous', matches: rows.slice(0, 3) };
}
```

10.2. Pipeline de RAG

backend/src/services/rag.ts:

```
import { prisma } from '../db/prisma.js';
import { embed, ask } from './gemini.js';

const TOP_K = 8;

export interface RagSource {
  driveFileId: string;
```

```

    fileName: string;
    pageNumber: number | null;
    score: number;
    webViewLink?: string;
  }

export interface RagResult {
  answer: string;
  sources: RagSource[];
  latencyMs: number;
  tokensIn?: number;
  tokensOut?: number;
}

interface ChunkRow {
  content: string;
  pageNumber: number | null;
  driveFileId: string;
  fileName: string;
  webViewLink: string | null;
  distance: number;
}

export async function runRag(question: string, clienteId: string): Promise<RagResult> {
  const t0 = Date.now();

  // 1. embedding da pergunta
  const queryVec = await embed(question);
  const vecLiteral = '[' + queryVec.join(',') + ']';

  // 2. busca por similaridade (cosine), filtrada pelo cliente
  const rows = await prisma.$queryRaw<ChunkRow[]>`
    SELECT
      c.content,
      c."pageNumber",
      f."driveFileId",
      f."fileName",
      f."webViewLink",
      (c.embedding <=> ${vecLiteral}::vector) AS distance
    FROM "DocChunk" c
    INNER JOIN "DriveFile" f ON f.id = c."driveFileId"
    WHERE c."clienteId" = ${clienteId}
    ORDER BY c.embedding <=> ${vecLiteral}::vector
    LIMIT ${TOP_K}
  `;

  if (rows.length === 0) {
    return {
      answer: 'Não há documentos indexados para este cliente ainda.',
      sources: [],
      latencyMs: Date.now() - t0,
    };
  }

  // 3. monta contexto e chama Gemini
  const context = rows
    .map(
      (r, i) =>
        `[Trecho ${i + 1}] (arquivo: ${r.fileName})${r.pageNumber ? ', página ${r.pageNumber}' : ''}\n${r.content}`,
    )
    .join('\n\n--\n\n');

  const answer = await ask(question, context);

  // 4. monta as fontes deduplicadas (um arquivo pode aparecer várias vezes)
  const seenFiles = new Set<string>();
  const sources: RagSource[] = [];
  for (const r of rows) {
    if (seenFiles.has(r.driveFileId)) continue;
    seenFiles.add(r.driveFileId);
    sources.push({
      driveFileId: r.driveFileId,
      fileName: r.fileName,
      pageNumber: r.pageNumber,
      score: 1 - r.distance, // converte distância em similaridade
      webViewLink: r.webViewLink ?? undefined,
    });
  }

  return { answer, sources, latencyMs: Date.now() - t0 };
}

```


10.3. Endpoint `/api/ask`

`backend/src/routes/ask.ts` :

```
import { Router } from 'express';
import { z } from 'zod';
import { prisma } from '../db/prisma.js';
import { resolveCliente } from '../services/clienteResolver.js';
import { runRag } from '../services/rag.js';
import { requireAuth } from '../middleware/auth.js';
import type { AuthRequest } from '../middleware/auth.js';

const router = Router();

const bodySchema = z.object({
  question: z.string().min(3).max(500),
  // Opcional: força um cliente específico (vindo do dashboard)
  clienteId: z.string().uuid().optional(),
});

router.post('/ask', requireAuth, async (req: AuthRequest, res) => {
  const parse = bodySchema.safeParse(req.body);
  if (!parse.success) {
    return res.status(400).json({ error: parse.error.flatten() });
  }
  const { question, clienteId: forcedId } = parse.data;
  const userId = req.user!.id;
  const t0 = Date.now();

  try {
    let clienteId = forcedId;

    if (!clienteId) {
      const resolution = await resolveCliente(question);

      if (resolution.status === 'ambiguous') {
        return res.json({
          type: 'ambiguous',
          message: 'Encontrei várias empresas com nome parecido. Qual delas?',
          options: resolution.matches,
        });
      }
      if (resolution.status === 'not_found') {
        return res.json({
          type: 'not_found',
          message: 'Não localizei essa empresa nas pastas do Drive. Confira o nome.',
        });
      }
      if (resolution.status === 'no_company_in_question') {
        return res.json({
          type: 'no_company',
          message: 'Por favor, mencione qual empresa-cliente sua pergunta se refere.',
        });
      }
      clienteId = resolution.matches[0].id;
    }

    const ragResult = await runRag(question, clienteId);

    await prisma.queryLog.create({
      data: {
        userId,
        clienteId,
        question,
        answer: ragResult.answer,
        sources: ragResult.sources as any,
        channel: 'DASHBOARD',
        latencyMs: Date.now() - t0,
      },
    });

    res.json({
      type: 'answer',
      answer: ragResult.answer,
      sources: ragResult.sources,
      clienteId,
      latencyMs: ragResult.latencyMs,
    });
  } catch (err: any) {
    console.error('Erro em /ask:', err);
  }
});
```

```
await prisma.queryLog.create({
  data: {
    userId,
    question,
    answer: '',
    sources: [] as any,
    channel: 'DASHBOARD',
    latencyMs: Date.now() - t0,
    errorMsg: err.message?.slice(0, 500),
  },
});
res.status(500).json({ error: 'Falha ao processar a pergunta' });
}
});

export default router;
```

11. Autenticação, Rotas e Servidor

11.1. Middleware de auth

backend/src/middleware/auth.ts :

```
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import { env } from '../config/env.js';
import { prisma } from '../db/prisma.js';

export interface AuthRequest extends Request {
  user?: { id: string; email: string };
}

export async function requireAuth(req: AuthRequest, res: Response, next: NextFunction) {
  const token = req.cookies?.access_token;
  if (!token) return res.status(401).json({ error: 'Não autenticado' });

  try {
    const payload = jwt.verify(token, env.JWT_SECRET) as { sub: string };
    const user = await prisma.appUser.findUnique({
      where: { id: payload.sub },
      select: { id: true, email: true, isActive: true },
    });
    if (!user || !user.isActive) {
      return res.status(401).json({ error: 'Usuário inativo' });
    }
    req.user = { id: user.id, email: user.email };
    next();
  } catch {
    return res.status(401).json({ error: 'Token inválido' });
  }
}
```

11.2. Rate limiting

backend/src/middleware/rateLimit.ts :

```
import rateLimit from 'express-rate-limit';

export const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5, // 5 tentativas a cada 15 min
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Muitas tentativas. Tente novamente em 15 minutos.' },
});

export const askLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 30, // 30 perguntas por minuto por IP
});

export const generalLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 120,
});
```

11.3. Rotas de autenticação

backend/src/routes/auth.ts :

```
import { Router } from 'express';
import bcrypt from 'bcrypt';
import jwt from 'jsonwebtoken';
import crypto from 'crypto';
import { authenticator } from 'otplib';
import { z } from 'zod';
import { prisma } from '../db/prisma.js';
import { env } from '../config/env.js';
```

```

import { loginLimiter } from '../middleware/rateLimit.js';

const router = Router();

const loginSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
  totp: z.string().optional(),
});

router.post('/login', loginLimiter, async (req, res) => {
  const parse = loginSchema.safeParse(req.body);
  if (!parse.success) {
    return res.status(400).json({ error: 'Dados inválidos' });
  }
  const { email, password, totp } = parse.data;

  const user = await prisma.appUser.findUnique({ where: { email } });
  if (!user || !user.isActive) {
    return res.status(401).json({ error: 'Credenciais inválidas' });
  }

  const ok = await bcrypt.compare(password, user.passwordHash);
  if (!ok) {
    return res.status(401).json({ error: 'Credenciais inválidas' });
  }

  // MFA obrigatório se configurado
  if (user.totpSecret) {
    if (!totp) return res.status(401).json({ error: 'TOTP obrigatório', mfa: true });
    const valid = authenticator.verify({ token: totp, secret: user.totpSecret });
    if (!valid) return res.status(401).json({ error: 'TOTP inválido' });
  }

  const accessToken = jwt.sign({ sub: user.id }, env.JWT_SECRET, {
    expiresIn: env.JWT_EXPIRES_IN,
  });
  const refreshToken = crypto.randomBytes(48).toString('hex');
  const refreshExpiry = new Date(Date.now() + 14 * 24 * 60 * 60 * 1000);

  await prisma.userSession.create({
    data: {
      userId: user.id,
      refreshToken,
      expiresAt: refreshExpiry,
      userAgent: req.headers['user-agent'] ?? null,
      ipAddress: req.ip ?? null,
    },
  });

  res
    .cookie('access_token', accessToken, {
      httpOnly: true,
      secure: env.COOKIE_SECURE,
      sameSite: env.COOKIE_SAMESITE,
      maxAge: 2 * 60 * 60 * 1000,
    })
    .cookie('refresh_token', refreshToken, {
      httpOnly: true,
      secure: env.COOKIE_SECURE,
      sameSite: env.COOKIE_SAMESITE,
      maxAge: 14 * 24 * 60 * 60 * 1000,
    })
    .json({ user: { id: user.id, email: user.email, name: user.name } });
});

router.post('/logout', async (req, res) => {
  const refreshToken = req.cookies?.refresh_token;
  if (refreshToken) {
    await prisma.userSession.updateMany({
      where: { refreshToken, revokedAt: null },
      data: { revokedAt: new Date() },
    });
  }
  res.clearCookie('access_token').clearCookie('refresh_token').json({ ok: true });
});

router.post('/refresh', async (req, res) => {
  const refreshToken = req.cookies?.refresh_token;
  if (!refreshToken) return res.status(401).json({ error: 'Sem refresh' });

  const session = await prisma.userSession.findUnique({
    where: { refreshToken },
    include: { user: true },
  });

```

```

});
if (!session || session.revokedAt || session.expiresAt < new Date()) {
  return res.status(401).json({ error: 'Sessão inválida' });
}

// Rotação: invalida o antigo e gera novo
const newRefresh = crypto.randomBytes(48).toString('hex');
await prisma.$transaction([
  prisma.userSession.update({
    where: { id: session.id },
    data: { revokedAt: new Date() },
  }),
  prisma.userSession.create({
    data: {
      userId: session.userId,
      refreshToken: newRefresh,
      expiresAt: new Date(Date.now() + 14 * 24 * 60 * 60 * 1000),
      userAgent: req.headers['user-agent'] ?? null,
      ipAddress: req.ip ?? null,
    },
  }),
]);

const newAccess = jwt.sign({ sub: session.userId }, env.JWT_SECRET, {
  expiresIn: env.JWT_EXPIRES_IN,
});

res
  .cookie('access_token', newAccess, {
    httpOnly: true,
    secure: env.COOKIE_SECURE,
    sameSite: env.COOKIE_SAMESITE,
    maxAge: 2 * 60 * 60 * 1000,
  })
  .cookie('refresh_token', newRefresh, {
    httpOnly: true,
    secure: env.COOKIE_SECURE,
    sameSite: env.COOKIE_SAMESITE,
    maxAge: 14 * 24 * 60 * 60 * 1000,
  })
  .json({ ok: true });
});

export default router;

```

11.4. Rotas de clientes e arquivos

backend/src/routes/clientes.ts :

```

import { Router } from 'express';
import { z } from 'zod';
import { prisma } from '../db/prisma.js';
import { requireAuth } from '../middleware/auth.js';

const router = Router();

// Lista com busca fuzzy
router.get('/clientes', requireAuth, async (req, res) => {
  const search = (req.query.search as string | undefined)?.trim() ?? '';
  const limit = Math.min(Number(req.query.limit) || 50, 200);

  if (search.length >= 2) {
    const rows = await prisma.$queryRaw<any[]>`
      SELECT id, "nomeEmpresa", "driveFolderId", cnpj,
        similarity(unaccent(lower("nomeEmpresa")), unaccent(lower(${search}))) AS score
      FROM "Cliente"
      WHERE unaccent(lower("nomeEmpresa")) % unaccent(lower(${search}))
      AND "isActive" = true
      ORDER BY score DESC
      LIMIT ${limit}
    `;
    return res.json(rows);
  }

  const all = await prisma.cliente.findMany({
    where: { isActive: true },
    orderBy: { nomeEmpresa: 'asc' },
    take: limit,
  });
  res.json(all);
});

router.get('/clientes/:id', requireAuth, async (req, res) => {
  const cliente = await prisma.cliente.findUnique({
    where: { id: req.params.id },
    include: {
      _count: { select: { files: true } },
    },
  });
  if (!cliente) return res.status(404).json({ error: 'Não encontrado' });
  res.json(cliente);
});

export default router;

```

backend/src/routes/files.ts :

```

import { Router } from 'express';
import { prisma } from '../db/prisma.js';
import { requireAuth } from '../middleware/auth.js';

const router = Router();

router.get('/clientes/:id/files', requireAuth, async (req, res) => {
  const { id } = req.params;
  const tipo = req.query.tipo as string | undefined; // pdf|docx|xlsx

  const where: any = { clienteId: id };
  if (tipo === 'pdf') where.mimeType = 'application/pdf';
  else if (tipo === 'docx') where.mimeType = { contains: 'wordprocessingml' };
  else if (tipo === 'xlsx') where.mimeType = { contains: 'spreadsheetml' };

  const files = await prisma.driveFile.findMany({
    where,
    orderBy: { modifiedTime: 'desc' },
    select: {
      id: true,
      driveFileId: true,
      fileName: true,
      mimeType: true,
      sizeBytes: true,
      modifiedTime: true,
      webViewLink: true,
      indexStatus: true,
      isScanned: true,
      pageCount: true,
    },
  });

  // BigInt não serializa direto em JSON
  res.json(
    files.map((f) => ({ ...f, sizeBytes: Number(f.sizeBytes) })),
  );
});

export default router;

```

11.5. Servidor principal

backend/src/index.ts :

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import cookieParser from 'cookie-parser';
import { env } from './config/env.js';
import { generalLimiter, askLimiter } from './middleware/rateLimit.js';
import authRoutes from './routes/auth.js';
import askRoutes from './routes/ask.js';
import clientesRoutes from './routes/clientes.js';
import filesRoutes from './routes/files.js';

const app = express();

app.set('trust proxy', 1); // detrás do Nginx
app.use(helmet());
app.use(
  cors({
    origin: env.PUBLIC_URL,
    credentials: true,
  }),
);
app.use(express.json({ limit: '1mb' }));
app.use(cookieParser());
app.use(generalLimiter);

app.get('/health', (_req, res) => res.json({ ok: true, time: new Date() }));

app.use('/api/auth', authRoutes);
app.use('/api', askLimiter, askRoutes);
app.use('/api', clientesRoutes);
app.use('/api', filesRoutes);

app.use((err: any, _req: any, res: any, _next: any) => {
  console.error('Erro não tratado:', err);
  res.status(500).json({ error: 'Erro interno' });
});

app.listen(env.PORT, () => {
  console.log(` API escutando em :${env.PORT}`);
});
```


12. Bot WhatsApp

O bot usa Baileys (provisório — migrar para Cloud API quando possível) e roteia mensagens pelo mesmo pipeline RAG.

`backend/src/services/whatsapp.ts` :

```
import {
  default as makeWASocket,
  useMultiFileAuthState,
  DisconnectReason,
  WAMessage,
} from '@baileys/baileys';
import { Boom } from '@hapi/boom';
import qrcode from 'qrcode';
import fs from 'fs/promises';
import { env } from '../../config/env.js';
import { prisma } from '../../db/prisma.js';
import { resolveCliente } from '../../clienteResolver.js';
import { runRag } from '../../rag.js';

const AUTH_DIR = '/root/saas_financeiro/baileys_auth';
const WHITELIST = new Set(env.WHATSAPP_WHITELIST);

// Sessão simples: lembra última pasta-cliente por número
// (para perguntas de follow-up tipo "e o telefone?")
const lastClienteByPhone = new Map<string, string>();

export async function startWhatsApp() {
  await fs.mkdir(AUTH_DIR, { recursive: true });
  const { state, saveCreds } = await useMultiFileAuthState(AUTH_DIR);

  const sock = makeWASocket({
    auth: state,
    printQRInTerminal: false,
  });

  sock.ev.on('creds.update', saveCreds);

  sock.ev.on('connection.update', async (update) => {
    const { connection, lastDisconnect, qr } = update;

    if (qr) {
      const qrPath = '/root/saas_financeiro/qr.png';
      await qrcode.toFile(qrPath, qr);
      console.log(`QR Code gerado em: ${qrPath}`);
      console.log('Acesse via SCP para escanear no celular.');
```

```

const phone = remoteJid.replace('@s.whatsapp.net', '');

// Whitelist
if (!WHITELIST.has(phone)) {
  console.log(` Mensagem ignorada de número não autorizado: ${phone}`);
  return;
}

// Identifica o usuário interno
const user = await prisma.appUser.findUnique({
  where: { phoneE164: phone },
});
if (!user) {
  await sock.sendMessage(remoteJid, {
    text: '⚠ Seu número não está cadastrado no sistema.',
  });
  return;
}

// Extrai texto (pode vir em vários formatos)
const text =
  msg.message.conversation ??
  msg.message.extendedTextMessage?.text ??
  msg.message.imageMessage?.caption ??
  '';

if (!text.trim()) {
  await sock.sendMessage(remoteJid, {
    text: ' Por favor, envie sua pergunta em texto.',
  });
  return;
}

// Verifica se é seleção numérica (resposta a ambiguidade)
const selection = await checkClienteSelection(phone, text);
let clienteId: string | undefined = selection ?? lastClienteByPhone.get(phone);

const t0 = Date.now();

try {
  if (!clienteId) {
    // Resolve cliente pela pergunta
    const resolution = await resolveCliente(text);

    if (resolution.status === 'ambiguous') {
      const options = resolution.matches
        .map((m, i) => `${i + 1}) ${m.nomeEmpresa}`)
        .join('\n');
      pendingSelections.set(phone, resolution.matches);
      await sock.sendMessage(remoteJid, {
        text:
          ` Encontrei várias empresas. Responda com o número:\n\n${options}\n\n` +
          `Depois mande sua pergunta original novamente.`,
      });
      return;
    }
    if (resolution.status === 'not_found') {
      await sock.sendMessage(remoteJid, {
        text:
          ' Não localizei essa empresa nas pastas. Confira o nome.',
      });
      return;
    }
    if (resolution.status === 'no_company_in_question') {
      await sock.sendMessage(remoteJid, {
        text:
          ' Mencione qual cliente sua pergunta se refere.\nEx: "Qual o CNPJ da Padaria do João?"',
      });
      return;
    }
  }
  clienteId = resolution.matches[0].id;
  lastClienteByPhone.set(phone, clienteId);

  await sock.sendMessage(remoteJid, {
    text: ` Buscando em: *${resolution.matches[0].nomeEmpresa}*...`,
  });
}

const result = await runRag(text, clienteId);

const sourcesText = result.sources
  .slice(0, 3)
  .map((s) => {
    const page = s.pageNumber ? ` (pág. ${s.pageNumber})` : '';

```

```

        return ` ${s.fileName}${page}`;
    })
    .join('\n');

const driveLink = result.sources[0]?webViewLink
? '\n\n ${result.sources[0].webViewLink}'
: '';

await sock.sendMessage(remoteJid, {
    text: `${result.answer}\n\n*Fontes:* \n${sourcesText}${driveLink}`,
});

await prisma.queryLog.create({
    data: {
        userId: user.id,
        clienteId,
        question: text,
        answer: result.answer,
        sources: result.sources as any,
        channel: 'WHATSAPP',
        latencyMs: Date.now() - t0,
    },
});
} catch (err: any) {
    console.error('Erro WhatsApp:', err);
    await sock.sendMessage(remoteJid, {
        text: '⚠ Tive um problema. Tente reformular a pergunta.',
    });
    await prisma.queryLog.create({
        data: {
            userId: user.id,
            clienteId,
            question: text,
            answer: '',
            sources: [] as any,
            channel: 'WHATSAPP',
            latencyMs: Date.now() - t0,
            errorMsg: err.message?.slice(0, 500),
        },
    });
}
}

// Estado em memória para seleção de cliente em caso de ambiguidade
const pendingSelections = new Map<string, any[]>();

async function checkClienteSelection(
    phone: string,
    text: string,
): Promise<string | null> {
    const trimmed = text.trim();
    // Aceita "1", "1]", "opção 1" etc.
    const match = trimmed.match(/^(?:opç[ãä]o\s*)?([1-5])/i);
    if (!match) return null;
    const idx = parseInt(match[1], 10) - 1;
    const pending = pendingSelections.get(phone);
    if (!pending || idx < 0 || idx >= pending.length) return null;

    const chosen = pending[idx];
    pendingSelections.delete(phone);
    lastClienteByPhone.set(phone, chosen.id);
    return chosen.id;
}

```

Para subir o bot junto com o servidor, adicione no `index.ts` :

```

import { startWhatsApp } from './services/whatsapp.js';

// Após app.listen(...)
startWhatsApp().catch((err) => console.error('WhatsApp falhou:', err));

```

Em produção, é melhor rodar o WhatsApp como processo PM2 separado para que reinicializações da API não derrubem a sessão.

13. Frontend (React + Vite + TypeScript)

13.1. Setup

frontend/package.json :

```
{
  "name": "saas-financeiro-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.7.7",
    "lucide-react": "^0.451.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.26.2"
  },
  "devDependencies": {
    "@types/react": "^18.3.10",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.1",
    "typescript": "^5.6.2",
    "vite": "^5.4.7"
  }
}
```

13.2. Cliente HTTP

frontend/src/api/client.ts :

```
import axios from 'axios';

export const api = axios.create({
  baseURL: '/api',
  withCredentials: true,
});

// Refresh automático no 401
let isRefreshing = false;
let queue: Array<() => void> = [];

api.interceptors.response.use(
  (r) => r,
  async (error) => {
    const original = error.config;
    if (
      error.response?.status === 401 &&
      !original._retry &&
      !original.url.includes('/auth/')
    ) {
      if (isRefreshing) {
        return new Promise((resolve) => {
          queue.push(() => resolve(api(original)));
        });
      }
      original._retry = true;
      isRefreshing = true;
      try {
        await api.post('/auth/refresh');
        queue.forEach((cb) => cb());
        queue = [];
        return api(original);
      } catch (e) {
        window.location.href = '/login';
        return Promise.reject(e);
      } finally {
        isRefreshing = false;
      }
    }
  }
)
```

```

    return Promise.reject(error);
  },
);

export interface Cliente {
  id: string;
  nomeEmpresa: string;
  driveFolderId: string;
  cnpj?: string;
}

export interface DriveFile {
  id: string;
  driveFileId: string;
  fileName: string;
  mimeType: string;
  sizeBytes: number;
  modifiedTime: string;
  webViewLink?: string;
  indexStatus: string;
  isScanned?: boolean;
  pageCount?: number;
}

export interface AskResponse {
  type: 'answer' | 'ambiguous' | 'not_found' | 'no_company';
  message?: string;
  answer?: string;
  sources?: Array<{
    driveFileId: string;
    fileName: string;
    pageNumber: number | null;
    score: number;
    webViewLink?: string;
  }>;
  options?: Cliente[];
  clienteId?: string;
}

export const listClientes = (search?: string) =>
  api.get<Cliente[]>('/clientes', { params: { search } }).then((r) => r.data);

export const getCliente = (id: string) =>
  api.get<Cliente>(`/clientes/${id}`).then((r) => r.data);

export const listFiles = (clienteId: string, tipo?: string) =>
  api
    .get<DriveFile[]>(`/clientes/${clienteId}/files`, { params: { tipo } })
    .then((r) => r.data);

export const ask = (question: string, clienteId?: string) =>
  api.post<AskResponse>('/ask', { question, clienteId }).then((r) => r.data);

export const login = (email: string, password: string, totp?: string) =>
  api.post('/auth/login', { email, password, totp }).then((r) => r.data);

export const logout = () => api.post('/auth/logout').then((r) => r.data);

```

13.3. Layout principal

frontend/src/App.tsx :

```

import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { Login } from '../pages/Login';
import { Dashboard } from '../pages/Dashboard';
import { ClientePage } from '../pages/Cliente';

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/login" element={<Login />} />
        <Route path="/" element={<Dashboard />} />
        <Route path="/cliente/:id" element={<ClientePage />} />
        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </BrowserRouter>
  );
}

```

13.4. Dashboard

frontend/src/pages/Dashboard.tsx :

```
import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Search, Folder, LogOut } from 'lucide-react';
import { Cliente, listClientes, logout } from '../api/client';
import { AskPanel } from '../components/AskPanel';

export function Dashboard() {
  const [clientes, setClientes] = useState<Cliente[]>([]);
  const [search, setSearch] = useState('');
  const [loading, setLoading] = useState(true);
  const navigate = useNavigate();

  useEffect(() => {
    const t = setTimeout(async () => {
      setLoading(true);
      try {
        const data = await listClientes(search);
        setClientes(data);
      } catch (err) {
        console.error(err);
      } finally {
        setLoading(false);
      }
    }, 250);
    return () => clearTimeout(t);
  }, [search]);

  async function handleLogout() {
    await logout();
    navigate('/login');
  }

  return (
    <div className="min-h-screen bg-slate-50 text-slate-900">
      <header className="bg-white border-b border-slate-200 sticky top-0 z-10">
        <div className="max-w-7xl mx-auto px-4 py-3 flex items-center justify-between">
          <h1 className="font-semibold text-lg">Bastos & Luz – Consulta Inteligente</h1>
          <button
            onClick={handleLogout}
            className="text-sm text-slate-600 hover:text-slate-900 flex items-center gap-2"
          >
            <LogOut size={16} /> Sair
          </button>
        </div>
      </header>

      <main className="max-w-7xl mx-auto px-4 py-6 grid grid-cols-1 lg:grid-cols-3 gap-6">
        <section className="lg:col-span-1">
          <div className="relative mb-3">
            <Search className="absolute left-3 top-2.5 text-slate-400" size={18} />
            <input
              className="w-full pl-10 pr-3 py-2 border border-slate-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500"
              placeholder="Buscar empresa..."
              value={search}
              onChange={(e) => setSearch(e.target.value)}
            />
          </div>

          <div className="bg-white border border-slate-200 rounded-lg overflow-hidden max-h-[70vh] overflow-y-auto">
            {loading && <div className="p-4 text-slate-500">Carregando...</div>}
            {!loading && clientes.length === 0 && (
              <div className="p-4 text-slate-500">Nenhum cliente encontrado.</div>
            )}
            {clientes.map((c) => (
              <button
                key={c.id}
                onClick={() => navigate(`/cliente/${c.id}`)}
                className="w-full text-left px-4 py-3 hover:bg-slate-50 border-b border-slate-100 flex items-center gap-3"
              >
                <Folder size={18} className="text-blue-500 shrink-0" />
                <div className="flex-1 min-w-0">
                  <div className="font-medium truncate">{c.nomeEmpresa}</div>
                  {c.cnpj && (
                    <div className="text-xs text-slate-500">{c.cnpj}</div>
                  )}
                </div>
              </button>
            )}
          </div>
        </section>
      </main>
    </div>
  );
}
```

```

    ))}
  </div>
</section>

  <section className="lg:col-span-2">
    <AskPanel />
  </section>
</main>
</div>
);
}

```

13.5. Página do cliente

frontend/src/pages/Cliente.tsx :

```

import { useEffect, useState } from 'react';
import { useParams, Link } from 'react-router-dom';
import { ArrowLeft, FileText, Sheet, FileType, ExternalLink, CheckCircle2, Loader2, AlertTriangle } from 'lucide-react';
import {
  Cliente,
  DriveFile,
  getCliente,
  listFiles,
} from '../api/client';
import { FilePreview } from '../components/FilePreview';
import { AskPanel } from '../components/AskPanel';

const ICON_BY_MIME = (mime: string) => {
  if (mime.includes('pdf')) return <FileText size={18} className="text-red-500" />;
  if (mime.includes('wordprocessingml')) return <FileText size={18} className="text-blue-600" />;
  if (mime.includes('spreadsheetml')) return <Sheet size={18} className="text-green-600" />;
  return <FileText size={18} className="text-slate-500" />;
};

const STATUS_BADGE = (status: string) => {
  if (status === 'INDEXED')
    return <span className="text-xs px-2 py-0.5 rounded bg-green-50 text-green-700 flex items-center gap-1"><CheckCircle2
size={12} />indexado</span>;
  if (status === 'FAILED')
    return <span className="text-xs px-2 py-0.5 rounded bg-red-50 text-red-700 flex items-center gap-1"><AlertTriangle
size={12} />falhou</span>;
  return <span className="text-xs px-2 py-0.5 rounded bg-amber-50 text-amber-700 flex items-center gap-1"><Loader2 size=
{12} className="animate-spin" />processando</span>;
};

export function ClientePage() {
  const { id } = useParams();
  const [cliente, setCliente] = useState<Cliente | null>(null);
  const [files, setFiles] = useState<DriveFile[]>([]);
  const [tipo, setTipo] = useState<string>('');
  const [selected, setSelected] = useState<DriveFile | null>(null);

  useEffect(() => {
    if (!id) return;
    getCliente(id).then(setCliente);
  }, [id]);

  useEffect(() => {
    if (!id) return;
    listFiles(id, tipo || undefined).then(setFiles);
  }, [id, tipo]);

  if (!cliente) return <div className="p-6">Carregando...</div>;

  return (
    <div className="min-h-screen bg-slate-50">
      <header className="bg-white border-b border-slate-200 sticky top-0 z-10">
        <div className="max-w-7xl mx-auto px-4 py-3 flex items-center gap-4">
          <Link to="/" className="text-slate-600 hover:text-slate-900">
            <ArrowLeft size={20} />
          </Link>
          <div>
            <div className="font-semibold">{cliente.nomeEmpresa}</div>
            <div className="text-xs text-slate-500">{cliente.cnpj ?? 'CNPJ não cadastrado'}</div>
          </div>
        </header>

        <main className="max-w-7xl mx-auto px-4 py-6 grid grid-cols-1 lg:grid-cols-12 gap-6">
          <section className="lg:col-span-4">

```

```

<div className="flex gap-2 mb-3">
  {[
    { v: '', label: 'Todos' },
    { v: 'pdf', label: 'PDF' },
    { v: 'docx', label: 'DOC' },
    { v: 'xlsx', label: 'XLS' },
  ].map((t) => (
    <button
      key={t.v}
      onClick={() => setTipo(t.v)}
      className={`px-3 py-1 rounded text-sm ${
        tipo === t.v
          ? 'bg-blue-500 text-white'
          : 'bg-white border border-slate-200 text-slate-700 hover:bg-slate-100'
      }`}
    >
      {t.label}
    </button>
  ))}
</div>

<div className="bg-white border border-slate-200 rounded-lg overflow-hidden max-h-[70vh] overflow-y-auto">
  {files.length === 0 && (
    <div className="p-4 text-slate-500 text-sm">Nenhum arquivo.</div>
  )}
  {files.map((f) => (
    <button
      key={f.id}
      onClick={() => setSelected(f)}
      className={`w-full text-left px-3 py-2 hover:bg-slate-50 border-b border-slate-100 flex items-center gap-2`}
    >
      {
        selected?.id === f.id ? 'bg-blue-50' : ''
      }
      <div className="flex-1 min-w-0">
        <div className="text-sm truncate">{f.fileName}</div>
        <div className="text-xs text-slate-500 flex items-center gap-2 mt-0.5">
          {STATUS_BADGE(f.indexStatus)}
          {f.isScanned && <span className="text-amber-600">OCR</span>}
        </div>
      </div>
    </button>
  ))}
</div>
</section>

<section className="lg:col-span-5">
  {selected ? (
    <FilePreview file={selected} />
  ) : (
    <div className="bg-white border border-dashed border-slate-300 rounded-lg p-12 text-center text-slate-500">
      Selecione um arquivo para visualizar
    </div>
  )}
</section>

<section className="lg:col-span-3">
  <AskPanel forceCliente={cliente} />
</section>
</main>
</div>
);
}

```

13.6. Preview de arquivo (iframe nativo do Drive)

frontend/src/components/FilePreview.tsx :


```
import { ExternalLink } from 'lucide-react';
import { DriveFile } from '../api/client';

export function FilePreview({ file }: { file: DriveFile }) {
  const previewUrl = `https://drive.google.com/file/d/${file.driveFileId}/preview`;
  const editUrl = file.webViewLink ?? `https://drive.google.com/open?id=${file.driveFileId}`;

  return (
    <div className="bg-white border border-slate-200 rounded-lg overflow-hidden flex flex-col h-[70vh]">
      <div className="px-4 py-2 border-b border-slate-200 flex items-center justify-between bg-slate-50">
        <div className="font-medium text-sm truncate">{file.fileName}</div>
        <a
          href={editUrl}
          target="_blank"
          rel="noreferrer"
          className="text-sm text-blue-600 hover:underline flex items-center gap-1 shrink-0"
        >
          > Abrir no Google <ExternalLink size={14} />
        </a>
      </div>
      <iframe
        src={previewUrl}
        className="flex-1 w-full border-0"
        allow="autoplay"
        title={file.fileName}
      />
    </div>
  );
}
```

13.7. Painele de consulta

frontend/src/components/AskPanel.tsx :

```
import { useState } from 'react';
import { Send, FileText } from 'lucide-react';
import { Cliente, ask, AskResponse } from '../api/client';

export function AskPanel({ forceCliente }: { forceCliente?: Cliente }) {
  const [question, setQuestion] = useState('');
  const [loading, setLoading] = useState(false);
  const [response, setResponse] = useState<AskResponse | null>(null);
  const [selectedFromAmbig, setSelectedFromAmbig] = useState<string | null>(null);

  async function submit(clienteId?: string) {
    if (!question.trim()) return;
    setLoading(true);
    try {
      const res = await ask(question, clienteId ?? forceCliente?.id);
      setResponse(res);
    } catch (err) {
      setResponse({ type: 'answer', answer: 'Erro ao consultar.' });
    } finally {
      setLoading(false);
    }
  }

  return (
    <div className="bg-white border border-slate-200 rounded-lg p-4 flex flex-col gap-3">
      <h2 className="font-semibold text-slate-900">
        Pergunte à IA
        {forceCliente && (
          <span className="text-xs font-normal text-slate-500 block">
            sobre: {forceCliente.nomeEmpresa}
          </span>
        )}
      </h2>
      <textarea
        className="w-full border border-slate-300 rounded p-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500"
        rows={3}
        placeholder={
          forceCliente
            ? 'Ex: Qual o CNPJ desse cliente?'
            : 'Ex: Qual o CNPJ da Padaria do João?'
        }
        value={question}
        onChange={(e) => setQuestion(e.target.value)}
      />
    </div>
  );
}
```


14. Scripts Auxiliares

14.1. Amostragem de OCR (rodar antes de começar)

`backend/scripts/sample-ocr.ts` :

```

import { driveClient } from '../src/config/google.js';
import { listClientFolders, listFilesInFolder, downloadFile } from '../src/services/drive.js';
import pdfParse from 'pdf-parse';
import { env } from '../src/config/env.js';

const SAMPLE_SIZE = 50;
const TEXT_THRESHOLD = 100;

function shuffle<T>(arr: T[]): T[] {
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}

async function main() {
  console.log(' Listando todas as pastas-cliente...');
  const folders = await listClientFolders(env.GOOGLE_DRIVE_ROOT_FOLDER_ID);
  console.log(` ${folders.length} pastas encontradas`);

  console.log(' Coletando todos os PDFs...');
  const allPdfs: Array<{ id: string; name: string }> = [];
  for (const folder of folders) {
    const files = await listFilesInFolder(folder.id);
    allPdfs.push(
      ...files
        .filter((f) => f.mimeType === 'application/pdf')
        .map((f) => ({ id: f.id, name: f.name })),
    );
  }
  console.log(` ${allPdfs.length} PDFs no acervo total`);

  if (allPdfs.length === 0) {
    console.log('Nenhum PDF encontrado.');
```

```

    return;
  }

  const sample = shuffle(allPdfs).slice(0, SAMPLE_SIZE);
  console.log(`\n Amostra de ${sample.length} arquivos:\n`);

  let scanned = 0;
  let textual = 0;
  let errors = 0;

  for (const file of sample) {
    try {
      const buffer = await downloadFile(file.id, 'application/pdf');
      const parsed = await pdfParse(buffer);
      const len = parsed.text.trim().length;
      const isScanned = len < TEXT_THRESHOLD;
      if (isScanned) scanned++;
      else textual++;
      console.log(
        `${isScanned ? ' ' : ' '}: ' ${file.name.slice(0, 60)}' - ${len} chars`,
      );
    } catch (err: any) {
      errors++;
      console.log(`\u2013 ${file.name}: erro ${err.message}`);
    }
  }

  console.log(`\n RESULTADO:');
  console.log(` Textual: ${textual} (${((textual / sample.length) * 100).toFixed(1)}%)`);
  console.log(` Escaneado: ${scanned} (${((scanned / sample.length) * 100).toFixed(1)}%)`);
  console.log(` \u2013 Erros: ${errors}`);

  const estimatedScanned = Math.round((scanned / sample.length) * allPdfs.length);
  const estimatedOcrCost = estimatedScanned * 5 * 0.0015; // ~5 p\u00e1ginas/arquivo @ $1.50/1000
  console.log(`\n Estimativa de PDFs escaneados no acervo: ~${estimatedScanned}`);
  console.log(` Custo aproximado de OCR \u00fanico: ~US$ ${estimatedOcrCost.toFixed(2)}`);
}

main()
  .catch(console.error)
  .finally(() => process.exit(0));

```

Rodar:

```
cd backend
npm run sample:ocr
```

14.2. Seed inicial de clientes

backend/scripts/seed-clientes.ts :

```
import { prisma } from '../src/db/prisma.js';
import { listClientFolders } from '../src/services/drive.js';
import { env } from '../src/config/env.js';

async function main() {
  const folders = await listClientFolders(env.GOOGLE_DRIVE_ROOT_FOLDER_ID);
  console.log(` ${folders.length} pastas para popular`);

  for (const folder of folders) {
    await prisma.cliente.upsert({
      where: { driveFolderId: folder.id },
      update: { nomeEmpresa: folder.name },
      create: { driveFolderId: folder.id, nomeEmpresa: folder.name },
    });
  }

  console.log(' Clientes populados');
}

main().finally(() => prisma.$disconnect());
```

14.3. Criação de usuários iniciais

Crie um script simples para cadastrar os 2 funcionários iniciais. Execute uma vez, depois apague:

backend/scripts/create-user.ts :

```
import bcrypt from 'bcrypt';
import { authenticator } from 'otplib';
import qrcode from 'qrcode';
import { prisma } from '../src/db/prisma.js';

async function main() {
  const email = process.argv[2];
  const phone = process.argv[3];
  const name = process.argv[4];
  const password = process.argv[5];

  if (!email || !phone || !name || !password) {
    console.error(
      'Uso: tsx scripts/create-user.ts <email> <phone E.164> <nome> <senha>',
    );
    process.exit(1);
  }

  const passwordHash = await bcrypt.hash(password, 12);
  const totpSecret = authenticator.generateSecret();

  const user = await prisma.appUser.create({
    data: {
      email,
      phoneE164: phone,
      name,
      passwordHash,
      totpSecret,
    },
  });

  const otpauth = authenticator.keyuri(email, 'Bastos & Luz', totpSecret);
  const qrPath = `/tmp/mfa-${user.id}.png`;
  await qrcode.toFile(qrPath, otpauth);

  console.log(` Usuário criado: ${user.email}`);
  console.log(` MFA QR salvo em: ${qrPath}`);
  console.log('Escaneie no Google Authenticator e DELETE o arquivo depois.');
```

```
main().finally(() => prisma.$disconnect());
```

14.4. Backup do Postgres (sai da VPS, criptografado)

backend/scripts/backup.sh :

```
#!/bin/bash
set -euo pipefail

BACKUP_DIR="/var/backups/saas_financeiro"
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
FILE="$BACKUP_DIR/saas_$TIMESTAMP.dump.gz.gpg"

DB_NAME="${DB_NAME:-saas_financeiro}"
DB_USER="${DB_USER:-saas_admin}"
PASSPHRASE_FILE="${BACKUP_GPG_PASSPHRASE_FILE:-/root/.backup_key}"
REMOTE="${BACKUP_REMOTE:-b2:bastosluz-backups}"

mkdir -p "$BACKUP_DIR"
chmod 700 "$BACKUP_DIR"

echo "[${date}] Iniciando backup..."

PGPASSWORD="$SDB_PASSWORD" pg_dump \
-U "$SDB_USER" -h localhost -d "$DB_NAME" \
-Fc --no-owner --no-acl \
| gzip -9 \
| gpg --batch --yes --passphrase-file "$PASSPHRASE_FILE" \
--symmetric --cipher-algo AES256 \
> "$FILE"

if [[ ! -s "$FILE" ]]; then
    echo "ERRO: arquivo de backup vazio" >&2
    exit 1
fi

SIZE=$(du -h "$FILE" | cut -f1)
echo "[${date}] Backup local OK ($SIZE). Enviando para $REMOTE..."

if ! rclone copy "$FILE" "$REMOTE/" --quiet; then
    echo "ERRO: upload remoto falhou" >&2
    # Continua mesmo assim - o backup local ainda foi feito
    exit 2
fi

# Retenção local: 7 dias. Remoto: configurar política de lifecycle no bucket.
find "$BACKUP_DIR" -type f -mtime +7 -name "*.dump.gz.gpg" -delete

echo "[${date}] Backup concluído com sucesso"
logger -t saas-backup "Backup OK em $TIMESTAMP"
```

Agendar via cron (`crontab -e`):

```
# Backup diário às 3h
0 3 * * * /root/saas_financeiro/backend/scripts/backup.sh >> /var/log/saas-backup.log 2>&1
```

Configurar alerta de falha (e-mail via `mailx` ou Discord/Slack webhook):

```
# Adicionar ao final do backup.sh:
trap 'curl -s -X POST $ALERT_WEBHOOK -d "Backup falhou em $(hostname): linha $LINENO" ' ERR
```

15. Configuração Nginx + TLS

/etc/nginx/sites-available/saas_financeiro.conf :

```
# Rate limit zones
limit_req_zone $binary_remote_addr zone=api_general:10m rate=60r/m;
limit_req_zone $binary_remote_addr zone=api_ask:10m rate=20r/m;
limit_req_zone $binary_remote_addr zone=api_login:10m rate=10r/m;

# Redirecionamento 80 -> 443
server {
    listen 80;
    listen [::]:80;
    server_name portal.bastoseluz.com.br;
    return 301 https://$host$request_uri;
}

# Servidor HTTPS
server {
    listen 443 ssl;
    listen [::]:443 ssl;
    http2 on;
    server_name portal.bastoseluz.com.br;

    # Certificados Let's Encrypt
    ssl_certificate /etc/letsencrypt/live/portal.bastoseluz.com.br/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/portal.bastoseluz.com.br/privkey.pem;

    # Protocolos e cifras (Mozilla "Modern", out/2024)
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305;
    ssl_prefer_server_ciphers off;
    ssl_ecdh_curve X25519:secp521r1:secp384r1;

    # Cache de sessão TLS
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 1d;
    ssl_session_tickets off;

    # OCSP Stapling
    ssl_stapling on;
    ssl_stapling_verify on;
    resolver 1.1.1.1 8.8.8.8 valid=300s;
    resolver_timeout 5s;

    # Security Headers
    # NOTA: ative `preload` no HSTS só depois de TUDO funcionando 100%.
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;
    # CSP: comece em Report-Only e endureça depois de testes
    add_header Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data: https; frame-src https://drive.google.com; connect-src 'self'" always;

    # Compressão
    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml application/xml text/javascript;
    gzip_min_length 1000;

    # Frontend (SPA)
    root /var/www/saas_financeiro/frontend;
    index index.html;

    client_max_body_size 25M;

    location / {
        try_files $uri $uri/ /index.html;
    }

    # API
    location /api {
        limit_req zone=api_general burst=20 nodelay;
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```

        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 10s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    location /api/ask {
        limit_req zone=api_ask burst=5 nodelay;
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s; # Gemini pode demorar
    }

    location /api/auth/login {
        limit_req zone=api_login burst=2 nodelay;
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Healthcheck público
    location = /health {
        proxy_pass http://127.0.0.1:5000;
        access_log off;
    }
}

```

Ativar e obter certificado:

```

sudo ln -s /etc/nginx/sites-available/saas_financeiro.conf /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
sudo certbot --nginx -d portal.bastoseluz.com.br

```


16. PM2 — Processos em Produção

deploy/ecosystem.config.js :

```
module.exports = {
  apps: [
    {
      name: 'saas-api',
      script: 'dist/index.js',
      cwd: '/root/saas_financeiro/backend',
      instances: 2,
      exec_mode: 'cluster',
      env: { NODE_ENV: 'production' },
      max_memory_restart: '500M',
    },
    {
      name: 'saas-indexer',
      script: 'dist/workers/indexFiles.js',
      cwd: '/root/saas_financeiro/backend',
      instances: 1,
      autorestart: true,
      max_memory_restart: '1G',
    },
    {
      name: 'saas-ocr',
      script: 'dist/workers/ocrPending.js',
      cwd: '/root/saas_financeiro/backend',
      instances: 1,
      autorestart: true,
    },
    {
      name: 'saas-whatsapp',
      script: 'dist/workers/whatsappStandalone.js',
      cwd: '/root/saas_financeiro/backend',
      instances: 1,
      autorestart: true,
      max_memory_restart: '500M',
    },
  ],
};
```

Sync do Drive como cron a cada 15 min:

```
*/15 * * * * cd /root/saas_financeiro/backend && /usr/bin/node dist/workers/syncDrive.js incremental >> /var/log/saas-sync.log 2>&1
```

17. Estimativa de Custos Operacionais

Premissas: 50 GB, 332 clientes, 2 usuários, ~200 perguntas/mês entre dashboard e WhatsApp.

Item	Volume	Custo unitário	Total estimado
Embeddings (indexação inicial)	~80k chunks de 800 tokens	US\$ 0,025/1M tokens	~ US\$ 2-5 (one-shot)
OCR Google Vision (estimativa otimista 20% escaneado)	~5.000 páginas	US\$ 1,50/1.000 páginas	~ US\$ 7,50 (one-shot)
OCR Google Vision (estimativa pessimista 60% escaneado)	~15.000 páginas	US\$ 1,50/1.000 páginas	~ US\$ 22 (one-shot)
Embeddings (novos arquivos diários, ~50/dia)	~5k chunks/mês	US\$ 0,025/1M	< US\$ 1/mês
Gemini Flash (consultas)	200 perguntas × ~5k input + 200 output tokens	US\$ 0,30/1M in, US\$ 2,50/1M out	~ US\$ 0,40/mês
Cloud Vision (novos PDFs escaneados/mês)	~300 páginas/mês	US\$ 1,50/1k	~ US\$ 0,45/mês
Postgres + storage	já incluso na VPS Hostinger	—	R\$ 0
Backblaze B2 (backup, ~500 MB/mês)	500 MB	US\$ 0,006/GB	< US\$ 0,01/mês

Total mensal recorrente: ~US\$ 2-5 (depois da indexação inicial). Indexação inicial fica em ~**US\$ 10-30** dependendo da proporção real de PDFs escaneados.

18. Roadmap de Implementação

Sprint 1 (semana 1) — Infraestrutura e descoberta

1. Apontar domínio para a VPS (Cloudflare ou Registro.br)
2. Certbot + Nginx com config completa acima → testar A+ no SSL Labs
3. Habilitar extensões `vector`, `pg_trgm`, `unaccent` no Postgres
4. Criar service account no GCP com escopo Drive + Vision + Gemini
5. Rodar `scripts/sample-ocr.ts` para descobrir % real de escaneados
6. Backup automatizado funcionando (com teste de restore em container)

Sprint 2 (semana 2) — Núcleo da indexação

1. Migrations do Prisma + vector column manual
2. Seed dos 332 clientes via `seed-clientes.ts`
3. Worker de sync incremental rodando via cron
4. Worker de indexação processando os 5 primeiros clientes manualmente
5. Validar qualidade do RAG com perguntas conhecidas em 5 clientes-piloto

Sprint 3 (semana 3) — Interface e auth

1. Login + MFA + cookies HttpOnly + refresh com rotação
2. Dashboard: lista de clientes com busca fuzzy
3. Página de cliente: lista de arquivos, preview iframe, AskPanel
4. Cadastrar os 2 usuários iniciais via `create-user.ts`

Sprint 4 (semana 4) — WhatsApp e indexação total

1. Baileys conectado, whitelist de 2 números
2. Fluxo conversacional completo (resolve cliente, ambiguidade, resposta)
3. Liberar worker de indexação para os 332 clientes
4. OCR rodando em paralelo para PDFs escaneados
5. Observabilidade básica: Sentry no backend, logs estruturados (pino)

Backlog (após go-live)

- Migração de Baileys para WhatsApp Cloud API (oficial)
- Atualização de cliente "cnpj" no banco via job que faz pergunta automática "qual o CNPJ?" para cada cliente e popula
- CI/CD com GitHub Actions (build + testes + deploy)
- Ambiente de staging
- Testes E2E críticos (login, ask, listagem)
- LGPD operacional: política de privacidade, ROPA, DPO, contratos de operador
- Histórico de perguntas no dashboard com re-ask
- Webhook real do Drive (push) ao invés de poll a cada 15min

19. Considerações Finais de Segurança e LGPD

- **Service account do Google:** dar acesso somente à pasta-raiz “Clientes”. Não dar permissões além do necessário (least privilege).
- **Chave do Gemini:** rotacionar a cada 90 dias.
- `.env`: nunca commitar. `chmod 600`. Considerar migrar para Doppler/Infisical.
- **Logs:** nunca logar o conteúdo bruto dos documentos. Só metadados (file_id, scores, latência).
- **LGPD:** como o sistema processa documentos com PII de clientes finais (CNPJs, CPFs, contratos), os papéis ficam:
 - Bastos & Luz = **controlador**
 - Você (desenvolvedor/operador da plataforma) = **operador**
 - Google (Drive, Vision, Gemini) = **suboperador**
 - Hostinger = **suboperador**
 - Contratos formais com cada um. Documentar tudo no ROPA.
- **Retenção:** definir prazo de retenção de `QueryLog` (sugestão: 5 anos para alinhar com obrigação contábil).
- **Direitos do titular:** ter procedimento para atender pedidos (acesso, correção, exclusão) — embora seja improvável vir de pessoa física diretamente, pode vir via cliente B2B.

Documento gerado em maio de 2026 como referência técnica para implementação. Todos os trechos de código foram escritos para servir como base — testes, refinamentos e adaptações específicas ficam a cargo do time de implementação.